

Important Question on Vector

Q. what is the default capacity of Vector?

The default capacity of Vector is 10.

Q. what is the load factor in Collection?

The load factor is the measure that decides when to increase the capacity collection.

Example: the load factor of Vector is 1 means 100% means when the capacity of Vector is 100% cross then Vector allocate double memory than its current capacity means Vector increase its capacity 100%.

Q. is Vector is Thread Safe?

Vector is a Thread safe collection because methods of Vector are synchronized so Vector is thread safe.

Q. Is Vector legacy Collection?

Yes Vector is legacy Collection Because Vector is not added in Collection framework before JDK 1.1 but later JDK 1.2 it is added in Collection Framework.

Q. Explain the all constructor of Vector?

Vector contain 4 types of Constructor

1) **Vector():**

2) **Vector(int initialCapacity)**

3) **Vector(int initialCapacity,int incrementalSize);**

4) **Vector(Collection collection);**

Vector (): this constructor is used for create the Vector with default capacity provided by Vector and the default capacity of Vector is 10.

Vector (int initialCapacity): this constructor is used for create a vector with a initial capacity provided by user.

Note: if user uses these two constructors and Vector cross its capacity then Vector allocate double memory then its current capacity if user want to set the increment capacity as per his choice then Vector contain third constructor.

Vector (int initialCapacity, int incrementalCapacity): this constructor contain two parameter

int initialCapacity: this parameter is used for set the initialCapacity

int incrementCapacity: this parameter decide how much capacity increment if current capacity cross.

Example

```
import java.util.*;
public class TestVectorApp
{ public static void main(String x[])
    {   Vector  v = new Vector(5,2);
        v.add(100);
        v.add(200);
        v.add(300);
        v.add(400);
        v.add(500);
        v.add(600);
        v.add(700);

        System.out.println("Now Capacity is "+v.capacity());
    }
}
```

```
}  
}
```

Q.what is the time complexity?

Time Complexity of an algorithm qualifies amount of time taken by an algorithm to runs a function of the length of input.

Q. what is the time complexity of Vector in Java?

- 1) Retrieving element from a specific position $O(1)$
- 2) Adding and Removing element from the end constant time complexity $O(1)$

Note: Adding and Removing element from other position is expensive learner $O(n-1)$ where n is the number of elements and i is the index of element added or removed these operations are more expensive because you have to shift all element at index i and higher over by one element.

Q. how does Vector grow internally in Java ?

A Vector will grow based on its capacity increment value new length of Vector is calculated as given below.

`newlength = capacityIncrement==0? existingSize+existingSize: existingSize+capacityIncrement;`

Once new size of Vector is calculated `Arrays.copyOf (existingArray, newlength)` is called the existing array and new length of array passed to the method. This method will create a new array of given length and copy data in to the new array.

Q. How to Remove Duplicates from ArrayList in Java?

ArrayList is the most popular implementation of the List interface from Java's Collection framework, but it allows duplicates. Though there is another collection called Set which is primarily designed to store unique elements, there are situations when you receive a List like ArrayList in your code and you need to ensure that it doesn't contain any duplicate before processing. Since with ArrayList you cannot guarantee uniqueness, there is no other choice but to remove repeated elements from ArrayList. The simplest approach to remove repeated objects from ArrayList is to copy them to a Set e.g. HashSet and then copy it back to ArrayList. This will remove all duplicates without writing any more code. One thing to noted is that, if original order of elements in ArrayList is important for you, as List maintains insertion order, you should use LinkedHashSet because HashSet doesn't provide any ordering guarantee.

Java Program to Remove duplicates from ArrayList

Here is our sample program to learn how to remove duplicates from ArrayList. The steps followed in the below example are:

- Copying all the elements of ArrayList to LinkedHashSet. Why we choose LinkedHashSet? Because it removes duplicates and maintains the insertion order.
- Emptying the ArrayList, you can use `clear()` method to remove all elements of ArrayList and start fresh.
- Copying all the elements of LinkedHashSet (non-duplicate elements) to the ArrayList.

Please find below the complete code:

```
import java.util.ArrayList;  
import java.util.LinkedHashSet;  
import java.util.List;  
import java.util.Set;  
public class ArrayListDuplicateDemo
```

```

{
    public static void main(String args[])
    {
        List<Integer> primes = new ArrayList<Integer>();
        primes.add(2);
        primes.add(3);
        primes.add(5);
        primes.add(7); //duplicate
        primes.add(7);
        primes.add(11);
        System.out.println("list of prime numbers : " + primes);
        Set<Integer> primesWithoutDuplicates = new LinkedHashSet<Integer>(primes);
        primes.clear();
        primes.addAll(primesWithoutDuplicates);
        System.out.println("list of primes without duplicates : " + primes);
    }
}

```

Output list of prime numbers: [2, 3, 5, 7, 7, 11]

List of primes without duplicates: [2, 3, 5, 7, 11]

Q. How to reverse ArrayList in Java with Example ?

You can reverse ArrayList in Java by using the reverse() method of java.util.Collections class. This is one of the many utility methods provided by the Collections class e.g. sort () method for sorting ArrayList. The Collections. reverse () method also accepts a List, so you not only can reverse ArrayList but also any other implementation of List interface e.g. LinkedList or Vector or even a custom implementation. This method has a time complexity of O(n) i.e. it runs on linear time because it uses ListIterator of the given list. It reverses the order of an element in the specified list.

By the way, you cannot reverse an ArrayList using this method if the specified ArrayList or its ListIterator doesn't support set() operation. It switches between two algorithms depending upon the size of List or if List implements RandomAccess interface like ArrayList.

If a number of elements in List are less than REVERSE_THRESHOLD, which is equal to 18 then it uses for loop for swapping elements otherwise it uses list iterator. If you want to learn more about *how the reverse() method of Collections works*, you can see it's code from JDK itself or in the next section

By the way, this is a typesafe generic method and you can use it to reverse Integer, String, Float or any kind of List in Java. You can also see the classic book Core Java Volume 1 - Fundamentals by Cay S. Horstmann to learn more about key classes of the Java Development Kit.

Java Program to reverse ArrayList in Java

You can see that we have added numbers on increasing order but after calling reverse() method, it prints them on decreasing order. Unlike popular misconception, Comparable or Comparator is not used while reversing ArrayList in Java. Though they are used if you want to sort Array in Java.

```
import java.util.ArrayList;
import java.util.Collections;
public class ArrayListReverseDemo
{ public static void main(String args[])
  { ArrayList<String> listOfInts = new ArrayList<>();
    listOfInts.add("1");
    listOfInts.add("2");
    listOfInts.add("3");
    listOfInts.add("4");
    listOfInts.add("5");
    System.out.println("Before Reversing : " + listOfInts);
    Collections.reverse(listOfInts);
    System.out.println("After Reversing : " + listOfInts);
  }
}
```

How Collections.reverse() method works in Java

Here is the code snippet from java.util.Collections class which you can use to *reverse an ArrayList* or any kind of List in Java. You can see that it uses set() method of List interface for swapping elements and that's why you cannot reverse a read-only ArrayList because it doesn't support set() operation.

Logic of Collections.reverse() method

Here is the logic and explanation of how the Collections.reverse() method works and how it reverse the given collection.

```
public static void reverse(List<?> list)
{ int size = list.size();
  if (size < REVERSE_THRESHOLD || list instanceof RandomAccess)
  {
    for (int i=0, mid=size>>1, j=size-1; i<mid; i++, j--)
      swap(list, i, j);
  }
  else
  { ListIterator fwd = list.listIterator();
    ListIterator rev = list.listIterator(size);
    for (int i=0, mid=list.size()>>1; i<mid; i++)
    { Object tmp = fwd.next();
      fwd.set(rev.previous()); rev.set(tmp);
    }
  }
}
```

```
}
```

Q. Difference between Array vs ArrayList in ?

Both Array and Array List are used to store elements, which can be either primitive or objects in case of Array and only objects in case of ArrayList in Java. The main *difference between Array vs ArrayList in Java* is the static nature of the Array and the dynamic nature of ArrayList. Once created you can not change the size of Array but ArrayList can re-size itself when needed. Another notable difference between ArrayList and Array is that Array is part of core Java programming and has special syntax and semantics support in Java, While ArrayList is part of the Collection framework along with other popular classes like Vector, Hashtable, HashMap or LinkedList.

Array vs ArrayList in Java

1) First and Major difference between Array and ArrayList in Java is that Array is a fixed-length data structure while ArrayList is a variable-length Collection class. You can not change the length of Array once created in Java but ArrayList re-size itself when gets full depending upon the capacity and load factor.

Since ArrayList is internally backed by Array in Java, any resize operation in ArrayList will slow down performance as it involves creating a new Array and copying content from the old array to the new array.

2) Another difference between Array and ArrayList in Java is that you can not use Generics along with Array, as Array instance knows about what kind of type it can hold and throws `ArrayStoreException` if you try to store type which is not convertible into the type of Array. ArrayList allows you to use Generics to ensure type safety.

3) You can also compare Array vs ArrayList on *How to calculate the length of Array or size of ArrayList*. All kinds of Array provides length variable which denotes the length of Array while ArrayList provides `size()` method to calculate the size of ArrayList in Java.

4) One more major difference between ArrayList and Array is that, you can not store primitives in ArrayList, it can only contain Objects. While Array can contain both primitives and Objects in Java. Though Autoboxing of Java 5 may give you an impression of storing primitives in ArrayList, it actually automatically converts primitives to Object. e.g.

5) Java provides add() method to insert an element into ArrayList and you can simply use the assignment operator to store element into Array e.g. In order to store Object to specified position use

```
Object[] objArray = new Object[10];  
objArray[1] = new Object();
```

6) One more difference on Array vs ArrayList is that you can create an instance of ArrayList without specifying size, Java will create Array List with default size but it's mandatory to provide the size of Array while creating either directly or indirectly by initializing Array while creating it. By the way, you can also initialize ArrayList while creating it.

Q. How to Synchronize ArrayList in Java with Example?

ArrayList is a very useful Collection in Java, I guess most used one as well but it is not synchronized. What this mean? It means you cannot share an instance of ArrayList between multiple threads if they are not just reading from it but also writing or updating elements. So *how can we synchronize ArrayList?* Well, we'll come to that in a second but did you thought why ArrayList is not synchronized in the first place? Since multi-threading is a core strength of Java and almost all Java programs have more than one thread, why Java designer does not make it easy for ArrayList to be used in such an environment?

The answer lies in performance, there is a performance cost associated with synchronization, and making ArrayList synchronized would have made it slower. So, they definitely thought about it and left ArrayList as non-synchronized to keep it fast, but at the same time they have provided easy ways to make it synchronized

The JDK Collections class has several methods to create synchronized List, Set and Map and we will use Collections.synchronizedList() method to make our ArrayList synchronized. This method accepts a List which could be any implementation of the List interface e.g. ArrayList, LinkedList, and returns a synchronized (thread-safe) list backed by the specified list. So you can also use this technique to make LinkedList synchronized and thread-safe in Java.

Synchronized List and Iteration

One of the main challenges of sharing ArrayList between multiple threads is how to deal with a situation where one thread is trying to access the element which is removed by other. If you are using methods like get(index) or remove(index) method to retrieve or remove elements then it's also possible that other thread may also be removing other elements.

This means you cannot call get(index) or remove(index) reliably without checking the size of the list first and then you also needs to provide extra synchronization between your call to size() and remove(int index).

In order to guarantee serial access, it is critical that all access to the backing list is accomplished through the returned list and it is imperative that the user manually synchronizes on the returned list when iterating over it as shown in the following sample code :

```
List list = Collections.synchronizedList(new ArrayList());  
...  
synchronized(list) {  
    Iterator i = list.iterator(); // Must be in synchronized block  
    while (i.hasNext())  
        foo(i.next());  
}
```

Java Program to Synchronize ArrayList

Here is the complete example of synchronizing an ArrayList in Java. If you look, we have created a List of String and added a couple of elements to it. Then we passed this ArrayList to Collections.synchronizedList() method, which returned a thread-safe, synchronized version of the backed list.

You can now safely share this list among multiple threads, but you need to be a little bit careful while retrieving or removing elements from ArrayList. In order to have safe access, you should use Iterator for getting and removing elements from the list and that too in a synchronized manner as shown in this example.

```
import java.util.ArrayList;  
import java.util.Collections;  
import java.util.Iterator;  
import java.util.List;  
public class SynchronizedArrayListDemo  
{ public static void main(String args[])  
    { // An ArrayList which is not synchronize  
        List<String> listOfSymbols = new ArrayList<String>();  
        listOfSymbols.add("RELIANCE");  
        listOfSymbols.add ("TATA");  
        listOfSymbols.add ("TECHMAH");  
        listOfSymbols.add ("HDFC");  
        listOfSymbols.add ("ICICI"); // Synchronizing ArrayList in Java  
        listOfSymbols = Collections.synchronizedList (listOfSymbols);  
        synchronized (listOfSymbols)  
        { Iterator<String> myIterator = listOfSymbols.iterator ();  
            while(myIterator.hasNext())
```

```
        { System.out.println(myIterator.next());  
        }  
    }  
}
```

Q. When to use ArrayList vs LinkedList in Java?

ArrayList and LinkedList are two popular concrete implementations of the List interface from Java's popular Collection framework. Being List implementation both ArrayList and LinkedList are ordered, the index-based and allows duplicate. Despite being from the same type of hierarchy there are a lot of differences between these two classes which makes them popular among Java interviewers.

The main difference between ArrayList vs LinkedList is that the former is backed by an array while the latter is based upon the linked list data structure, which makes the performance of add(), remove(), contains(), and iterator() different for both ArrayList and LinkedList. The difference between ArrayList and LinkedList is also an important Java collection interview question, as much popular as Vector vs ArrayList or HashMap vs HashSet in Java. Sometimes this is also asked as for when to use LinkedList and when to use ArrayList in Java.

In this Java collection tutorial, we will compare LinkedList vs ArrayList on various parameters which will help us to decide when to use ArrayList over LinkedList in Java. We will not focus on the array and linked list data structure much, which is subject to data structure and algorithm, we'll only focus on the Java implementations of these data structures which are ArrayList and LinkedList.

When to use ArrayList vs LinkedList in Java

Before comparing differences between ArrayList and LinkedList, let's see What is common between ArrayList and LinkedList in Java:

- 1) Both ArrayList and LinkedList are an implementation of the List interface, which means you can pass either ArrayList or LinkedList if a method accepts the java.util.List interface.
- 2) Both ArrayList and LinkedList are not synchronized, which means you cannot share them between multiple threads without external synchronization. See here to know.
- 3) ArrayList and LinkedList are ordered collection e.g. they maintain insertion order of elements i.e. the first element will be added to the first position.
- 4) ArrayList and LinkedList also allow duplicates and null, unlike any other List implementation e.g. Vector.
- 5) An iterator of both LinkedList and ArrayList are fail-fast which means they will throw ConcurrentModificationException if a collection is modified structurally once the Iterator is created. They are different than CopyOnWriteArrayList whose Iterator is fail-safe.

Difference between LinkedList and ArrayList in Java

Now let's see some differences between ArrayList and LinkedList and when to use ArrayList and LinkedList in Java.

1. Underlying Data Structure

The first difference between ArrayList and LinkedList comes with the fact that ArrayList is backed by Array while LinkedList is backed by LinkedList. This will lead to further differences in performance.

2. LinkedList implements Deque

Another difference between ArrayList and LinkedList is that apart from the List interface, LinkedList also implements the Deque interface, which provides first in first out operations for add() and poll() and several other Deque functions.

3. Adding elements in ArrayList

Adding an element in ArrayList is $O(1)$ operation if it doesn't trigger re-size of Array, in which case it becomes $O(\log(n))$, On the other hand, appending an element in LinkedList is $O(1)$ operation, as it doesn't require any navigation.

4. Removing an element from a position

In order to remove an element from a particular index e.g. by calling remove(index), ArrayList performs a copy operation which makes it close to $O(n)$ while LinkedList needs to traverse to that point which also makes it $O(n/2)$, as it can traverse from either direction based upon proximity.

5. Iterating over ArrayList or LinkedList

Iteration is the $O(n)$ operation for both LinkedList and ArrayList where n is a number of an element.

6. Retrieving element from a position

the get(index) operation is $O(1)$ in ArrayList while its $O(n/2)$ in LinkedList, as it needs to traverse till that entry. Though, in Big O notation $O(n/2)$ is just $O(n)$ because we ignore constants there.

7. Memory

LinkedList uses a wrapper object, Entry, which is a static nested class for storing data and two nodes next and previous while ArrayList just stores data in Array. So memory requirement seems less in the case of ArrayList than LinkedList except for the case where Array performs the re-size operation when it copies content from one Array to another.

If Array is large enough it may take a lot of memory at that point and trigger Garbage collection, which can slow response time.

From all the above differences between ArrayList vs LinkedList, It looks like ArrayList is the better choice than LinkedList in almost all cases, except when you do a frequent add () operation than remove (), or get ().

It's easier to modify a linked list than ArrayList, especially if you are adding or removing elements from start or end because the linked list internally keeps references of those positions and they are accessible in $O(1)$ time.

In other words, you don't need to traverse through the linked list to reach the position where you want to add elements, in that case, addition becomes an $O(n)$ operation. For example, inserting or deleting an element in the middle of a linked list

Q. Difference between ArrayList and HashSet in Java?

1. First and most important difference between ArrayList and HashSet is that ArrayList implements List interface while HashSet implements Set interface in Java.

2. Another difference between ArrayList and HashSet is that ArrayList allow duplicates while HashSet doesn't allow duplicates. This is the side effect of first difference and property of implementing List and Set interface.

3. The differences between ArrayList and HashSet is that ArrayList is an ordered collection and maintains insertion order of elements while HashSet is an unordered collection and doesn't maintain any order.

4. The difference between ArrayList and HashSet is that ArrayList is backed by an Array while HashSet is backed by a HashMap instance

5. Fifth difference between HashSet and ArrayList is that it's index-based you can retrieve objects by calling get(index) or remove objects by calling remove(index) while HashSet is completely object-based. HashSet also doesn't provide the get() method.

Similarities ArrayList and HashSet

1) Both ArrayList and HashSet are non synchronized collection classes and not meant to be used in multi-threading and concurrent environments. You can make ArrayList and HashSet synchronized by Using Collections.synchronizedCollection () just like we Make ArrayList and HashSet read-only other days

2) Both ArrayList and HashSet can be traversed using Iterator. This is in fact a preferred way if you want to perform operations on all elements.

3) Iterator of ArrayList and HashSet both are fail-fast, i.e. they will throw ConcurrentModificationException if ArrayList or HashSet is modified structurally once Iterator has been created.

Q. Difference between Vector and ArrayList in Java?

ArrayList and Vector are the two most widely used Collection classes in Java and are used to store objects in an ordered fashion. Every Java programmer which is introduced to Java Collection Framework either started with Vector or ArrayList

Vector vs ArrayList in Java

1. Synchronization

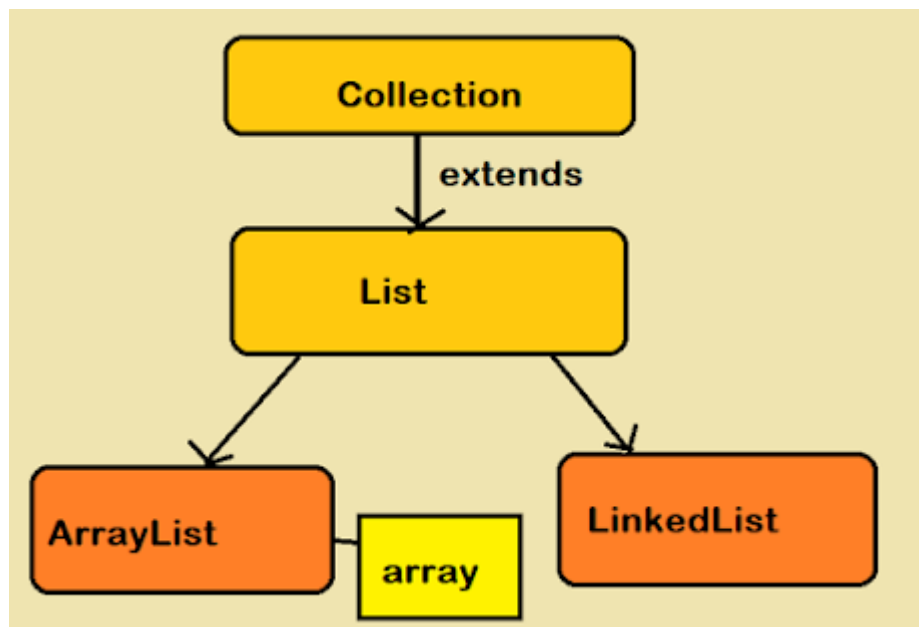
The first and most common *difference between Vector vs ArrayList* is that Vector is synchronized and thread-safe while ArrayList is neither Synchronized nor thread-safe. Now, What does that mean? It means if multiple thread try to access Vector same time they can do that without compromising Vector's internal state. Same is not true in case of ArrayList as methods like add(), remove() or get() is not synchronized.

2. Speed

The second major difference on Vector vs ArrayList is Speed, which is directly related to previous difference. Since Vector is synchronized, its slow and ArrayList is not synchronized its faster than Vector.

3. Legacy Class

The third difference on Vector vs ArrayList is that Vector is a legacy class and initially it was not part of the Java Collection Framework. From Java 1.4 Vector was retrofitted to implement List interface and become part of Collection Framework.



Q. How to sort ArrayList in Java?

Sorting ArrayList in Ascending Order in Java

First, let's see how to sort an array in ascending order in Java using the `Collections.sort()` method. This method is overloaded, which means you can sort the ArrayList in natural order by leveraging the default comparator, which sorts the list in the natural order, and you can use the `Collections.sort(list, Comparator)` to sort the ArrayList in custom order defined by Comparator. Since we are sorting ArrayList of String which implements a Comparable method, we can use the first method to sort the ArrayList string into the natural order or ascending order. You can also use this method to sort an ArrayList of Integer into increasing order because that's the default order for an Integer object.

```
import java.util.*;
```

```
class Main
```

```
{ public static void main(String[] args)
```

```
{ List<Integer> listofYears = new ArrayList<Integer>();
```

```
listofYears.add(2021);
```

```
listofYears.add(2019);
```

```
listofYears.add(2018);
```

```
listofYears.add(2020); // print the ArrayList before sorting
```

```
System.out.println("ArrayList before sorting");
```

```
System.out.println(listofYears); // sorting an ArrayList of Integer in ascending order
```

```
Collections.sort(listofYears); // print the ArrayList after sorting
```

```
System.out.println("ArrayList after sorting");
```

```
        System.out.println(listofYears);  
    }  
}
```

Q. Difference between ArrayList and HashMap in Java?

1. The first difference between ArrayList and HashMap is that ArrayList implements a List interface while HashMap implements Map interface in Java.
2. The second difference between ArrayList and HashMap is that ArrayList only stores one object while HashMap stores two objects key and value.
3. The third difference between HashMap and ArrayList is that keys of HashMap must implement equals and hashCode method correctly, ArrayList doesn't have that requirement but its good to have that because contains() method of ArrayList will use the equals() method to see if that object already exists or not.
4. The fourth difference between HashMap and ArrayList is that ArrayList maintains the order of objects, in which they are inserted while HashMap doesn't provide any ordering guarantee.
5. Another difference between ArrayList and HashMap is that ArrayList allows duplicates but HashMap doesn't allow duplicates key though it allows duplicate values.
6. ArrayList get(index) method always gives an O(1) performance but HashMap get(key) can be O(1) in the best case and O(n) in the worst case.

Q. Java program to get SubList from ArrayList – Example?

Sometimes we need *subList from ArrayList in Java*. For example, we have an ArrayList of 10 objects and we only need 5 objects or we need an object from index 2 to 6, these are called subList in Java. Java collection API provides a method to *get SubList from ArrayList*. In this Java tutorial, we will see an example of getting SubList from ArrayList in Java. In this program, we have an ArrayList which contains 4 String objects. Later we call ArrayList.subList () method to get part of that List

Example with Source code

```
import java.util.ArrayList;  
import java.util.List;  
public class GetSubListExample {  
    public static void main(String[] args) {  
        ArrayList arrayList = new ArrayList();  
        //Add elements to Arraylist  
        arrayList.add("Java");  
        arrayList.add("C++");  
        arrayList.add("PHP");  
        arrayList.add("Scala");  
        lst = arrayList.subList(1,3);  
        System.out.println("Sub list contains : ");  
        for(int i=0; i<lst.size() ; i++)  
            System.out.println(lst.get(i));  
    }  
}
```

```

//remove one element from sub list
Object obj = lst.remove(0);
System.out.println(obj + " is removed from sub list");
//print original ArrayList
System.out.println("After removing " + obj + " from sub list, original ArrayList contains : ");
for(int i=0; i<arrayList.size() ; i++)
    System.out.println(arrayList.get(i));
}
}

```

Q. Array length vs ArrayList Size in Java?

One of the confusing parts in learning Java for a beginner to understand how to find the length of array and ArrayList in Java? The main reason for the confusion is an inconsistent way of calculating the length between two. Calling size() method on arrays and length, or even length() on ArrayList is a common programming error made by beginners. The main reason for the confusion is the special handling of an array in Java. Java native arrays have built-in length attribute but no size() method while the Java library containers, known as Collection classes like ArrayList<>, Vector<>, etc, all have a size() method. There is one more thing which adds to this confusion, that is capacity, at any point capacity of any collection class is the maximum number of elements collection can hold. the size of collection must be less than or equal to its capacity.

Though in reality, collection resize themselves even before it reaches its capacity, controlled by load factor. I have mentioned this before on my post difference between ArrayList and Array in Java, if you not read it already, you may find some useful detail there as well.

So, use length attribute to get number of elements in a array, also known as length, and for same thing in Collection classes e.g. ArrayList, Vector, use size() method. To give you more context, consider following lines of code, can you spot the error, which is bothering our beginner friend:

```

import java.util.*;
public class ArrayListTest
{ public static void main(String[] args)
{   ArrayList<String> arrList = new ArrayList<String>();
    String[] items = { "One", "Two", "Three", "Four", "Five" };
    for(String str: items){
        arrList.add(str);
    }
    int size = items.size();
    System.out.println (size);
}
}
ArrayList("First", "Second", "Third", "Fourth", "Fifth");

```

It's handy way to declare and initialize ArrayList in same place, similar to array in Java. So next time don't confuse between length and size(), array are special object in Java and has attribute called length, which specifies number of buckets in array, while ArrayList is a Collection class, which inherit size() method, which returns number of elements inside Collection. Remember, size is different than capacity of collection. At any point, size <= capacity of collection.

Q. What is CopyOnWriteArrayList in Java ?

CopyOnWriteArrayList is a concurrent Collection class introduced in Java 5 Concurrency API along with its popular cousin ConcurrentHashMap in Java. CopyOnWriteArrayList implements List interface like ArrayList, Vector, and LinkedList but its a thread-safe collection and it achieves its thread-safety in a slightly different way than Vector or other thread-safe collection class. As the name suggests CopyOnWriteArrayList creates a copy of underlying ArrayList with every mutation operation e.g. add, remove, or when you set values. That's why it is only suitable for a small list of values which are read frequently but modified rarely e.g. a list of configurations.

Normally CopyOnWriteArrayList is very expensive because it involves costly Array copy with every writes operation but it's very efficient if you have a List where Iteration outnumbers mutation e.g. you mostly need to iterate the ArrayList and don't modify it too often.

Iterator of CopyOnWriteArrayList is fail-safe and doesn't throw ConcurrentModificationException even if underlying CopyOnWriteArrayList is modified once Iteration begins because Iterator is operating on a separate copy of ArrayList. Consequently, all the updates made on CopyOnWriteArrayList is not available to Iterator

Difference between CopyOnWriteArrayList and ArrayList in Java.

- 1) First and foremost difference between CopyOnWriteArrayList and ArrayList in Java is that CopyOnWriteArrayList is a thread-safe collection while ArrayList is not thread-safe and cannot be used in the multi-threaded environment.
- 2) The second difference between ArrayList and CopyOnWriteArrayList is that Iterator of ArrayList is fail-fast and throw ConcurrentModificationException once detect any modification in List once iteration begins but Iterator of CopyOnWriteArrayList is fail-safe and doesn't throw ConcurrentModificationException.
- 3) The third difference between CopyOnWriteArrayList vs ArrayList is that Iterator of former doesn't support remove operation while Iterator of later supports remove() operation. If you want to learn more about collections, I suggest you go through Complete Java MasterClass, one of the best Java course on Udemy.

```
import java.util.Iterator;
import java.util.concurrent.CopyOnWriteArrayList;
public class CopyOnWriteArrayListExample
{
```

```

public static void main(String args[])
{
    CopyOnWriteArrayList<String> threadSafeList = new CopyOnWriteArrayList<String>();
    threadSafeList.add("Java");
    threadSafeList.add("J2EE");
    threadSafeList.add("Collection");

    Iterator<String> failSafeIterator = threadSafeList.iterator();
    while(failSafeIterator.hasNext()){
        System.out.printf("Read from CopyOnWriteArrayList : %s %n", failSafeIterator.next());
        failSafeIterator.remove(); //not supported in CopyOnWriteArrayList in Java
    }
}

```

Q. Ways to Remove Elements/Objects From ArrayList in Java ?

There are *two ways to remove objects from ArrayList in Java*, first, by using the `remove()` method, and second by using `Iterator`. `ArrayList` provides overloaded `remove()` method, one accepts the index of the object to be removed i.e. `remove(int index)`, and the other accept objects to be removed, i.e. `remove(Object obj)`. The rule of thumb is, If you know the index of the object, then use the first method, otherwise use the second method. By the way, you must remember to use `ArrayList` remove methods, only when you are not iterating over `ArrayList` if you are iterating then use `Iterator`. `Remove()` method, failing to do so may result in `ConcurrentModificationException` in Java

Another gotcha can have occurred due to autoboxing. If you look closely that two remove methods, `remove(int index)` and `remove(Object obj)` are indistinguishable if you are trying to remove from an `ArrayList` of `Integers`.

Code Example To Remove Elements from ArrayList

Let's test the above theory with a simple code example of `ArrayList` with `Integers`. The following program has an `ArrayList` of `Integers` containing 1, 2, and 3 i.e. [1, 2, 3], which corresponds exactly to the index.

Suppose you have three objects in `ArrayList` i.e. [1,2,3] and you want to remove the second object, which is 2. You may call `remove(2)`, which is actually a call to `remove(Object)` if consider autoboxing, but will be interpreted as a call to remove 3rd element, by interpreting as `remove(index)`.

Q. How to Create Read Only, Unmodifiable ArrayList in Java?

You can create read-only Collection by using `Collections.unmodifiableCollection()` utility method. it returns an unmodifiable or read-only view of Collection in which you cannot perform any operation which will change the collection like `add()`, `remove()` and `set()` either directly or while iterating using `Iterator` or `ListIterator`. It will throw `UnsupportedOperationException` whenever you try to modify the List.

One of the common misconceptions around read-only `ArrayList` is that you can create read-only `ArrayList` by using `Arrays.asList(String{})`, which is apparently not true as this method only return a fixed-size list.

You cannot perform add() and remove() but the set() method is still allowed which can change the contents of ArrayList. Collections class also provides a different method to make List and Set read-only. In this Java tutorial, we will learn *How to make any collection read only* and How to create a fixed size List as well.

Example

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collection;
import java.util.Collections;
import java.util.List;
public class ReadOnlyCollection {

    public static void main(String args[])
    {
        Collection readOnlyCollection = Collections.unmodifiableCollection(new ArrayList<String>());
        ArrayList readableList = new ArrayList();
        readableList.add("Jeffrey Archer");
        readableList.add("Khalid Hussain");
        List unmodifiableList = Collections.unmodifiableList(readableList);
        unmodifiableList.add("R.K. Narayan");
        unmodifiableList.remove(0);
        unmodifiableList.set(0, "Anurag Kashyap");
        fixedLengthList.set(0, "J.K. Rowling");
        System.out.println(fixedLengthList.get(0));
    }
}
```

Differentiate between Iterator and Enumeration?

Iterator

- It can traverse both legacy as well as non legacy elements.
- It is slower than Enumeration.
- It can perform remove operations while traversing the collection.
- It is fail-fast.

Enumeration

- It can traverse only legacy elements.
- It is faster than an Iterator.
- It can perform only traverse operations on the collection.
- It is not a fail-fast.

What is Fail-fast and Fail-safe iterations in Java Collections?

Using iterations we can traverse over the collections objects. The iterators can be either fail-safe or fail-fast. *Fail-safe* iterators means they will not throw any exception even if the collection is modified while iterating over it. Whereas *Fail-fast* iterators throw an exception(*ConcurrentModificationException*) if the collection is modified while iterating over it.

Consider an example:

```
ArrayList<Integer> integers = new ArrayList<>();
integers.add(1);
integers.add(2);
integers.add(3);
Iterator<Integer> itr = integers.iterator();
while (itr.hasNext()) {
    Integer a = itr.next();
    integers.remove(a);
}
```

As ArrayLists are fail-fast above code will throw an exception.

First a will have value = 1, and then 1 will be removed in same iteration.

Next when a will try to get next(), as the modification is made to the list, it will throw an exception here.

However if we use an fail-safe collection e.g. CopyOnWriteArrayList then no exception will occur:

```
List integers = new CopyOnWriteArrayList();
integers.add(1);
integers.add(2);
integers.add(3);
Iterator itr = integers.iterator();
while (itr.hasNext()) {
    Integer a = itr.next();
    integers.remove(a);
}
```

Here if we print the element a, then all the elements will be printed.

Fail-Fast Iterators internal working:

Every fail fast collection has a **modCount** field, to represent how many times the collection has changed/modified.

So at every modification of this collection we increment the modCount value. For example the modCount is incremented in below cases:

1. When one or more elements are removed.
2. When one or more elements are added.
3. When the collection is replaced with other collection.
4. When the collection is sorted.

So everytime there is some change in the collection structure, the mod count is incremented.

Now the iterator stores the modCount value in the initialization as below:

```
int expectedModCount = modCount;
```

Now while the iteration is going on, expectedModCount will have old value of modCount.

If there is any change made in the collection, the modCount will change and then an exception is thrown using:

```
if (modCount != expectedModCount)
    throw new ConcurrentModificationException();
```

This code is used in most of the iterator methods e.g.

1. next()
2. remove()
3. add()

So if we make any changes to the collection, the modCount will change, and expectedModCount will not be hence equal to the modCount. Then if we use any of the above methods of iterator, the ConcurrentModificationException will be thrown.

Note: If we remove/add the element using the remove() or add() of iterator instead of collection, then in that case no exception will occur. It is because the remove/add methods of iterators call the remove/add method of collection internally, and also it reassigns the expectedModCount to new modCount value

```
ArrayList.this.remove(lastRet);
cursor = lastRet;
lastRet = -1;
expectedModCount = modCount;andArrayList.this.add(i, e);
expectedModCount = modCount;
```

So the below code is safe as we are removing the element from the iterator here:

```
Iterator<Integer> itr = integers.iterator();
while (itr.hasNext()) {
    if (itr.next() == 2) {
```

```
// will not throw Exception  
itr.remove();  
}  
}
```

Whereas the below code will throw an exception as we are removing the element from the collection here:

```
Iterator<Integer> itr = integers.iterator();  
while (itr.hasNext()) {  
    if (itr.next() == 3) {  
        // will throw Exception on  
        // next call of next() method  
        integers.remove(3);  
    }  
}
```

Fail-Safe Iterators internal working:

Unlike the fail-fast iterators, these iterators traverse over the clone of the collection. So even if the original collection gets structurally modified, no exception will be thrown. E.g. in case of CopyOnWriteArrayList the original collections is passed and is stored in the iterator:

```
public Iterator<E> iterator() {  
    return new COWIterator<E>(getArray(), 0);  
}
```

here iterator() method returns the iterator of the CopyOnWriteArrayList. As we can see, it passes the getArray() in the constructor of the iterator. This getArray() has all the collection elements. Now the iterator(COWIterator here) will save this to traverse upon as:

```
COWIterator(Object[] elements, int initialCursor) {  
    cursor = initialCursor;  
    snapshot = elements;  
}
```

So the original collection elements are saved in the snapshot field variable. So all the iterator methods will work on this snapshot method. So even if there is any change in

the original collection, no exception will be thrown. But note the the iterator will not reflect the latest state of the collection

```
Iterator<Integer> itr = integers.iterator();
while (itr.hasNext()) {
    int a = itr.next();
    if (a == 1) {
        integers.remove(Integer.valueOf(a));
    }
    System.out.print(a);
}
```

So as we are removing the element from the collection. the collection now has only elements **2 and 3** in it. But the iterator will print all the elements **1,2,3** because it traverses over the snapshot of the collection elements. We can print the collection elements after the above code. It will print only **2,3** as:

```
Iterator<Integer> itr = integers.iterator();
while (itr.hasNext()) {
    int a = itr.next();
    System.out.print(a);
}
```

Q.What is the diff between Iterator and ListIterator in java

Iterator	ListIterator
Can traverse elements present in Collection only in the forward direction.	Can traverse elements present in Collection both in forward and backward directions.
Helps to traverse Map, List and Set.	Can only traverse List and not the other two.
Cannot modify or replace elements present in Collection	We can modify or replace elements with the help of set(E e)
Cannot add elements and it throws ConcurrentModificationException.	Can easily add elements to a collection at any time.
Certain methods of Iterator are next(), remove() and hasNext().	Certain methods of ListIterator are next(), previous(), hasNext(), hasPrevious(), add(E e).

Q. How to describe Java Linked List in interview?

- Java LinkedList is the Doubly-linked list implementation of the list interface
- LinkedList can be iterated in reverse direction using descendingIterator()
- LinkedList can contain duplicate elements.
- LinkedList maintains insertion order.
- LinkedList is not synchronized.
- LinkedList manipulation is fast because no shifting needs to be occurred.
- LinkedList can be used as list, stack or queue.

Q. What is the performance of different linked list operations in terms of big 'O' notation?

Operations	Access	Search	Insertion	Deletion
Single Linked list	O(n) Slow	O(n) Slow	O(1) Fast	O(1) Fast
Doubly Linked list	O(n) Slow	O(n) Slow	O(1) Fast	O(1) Fast

Q. What are advantages and disadvantages of linked lists?**Advantages of linked lists in Java:**

- Linked lists have constant-time insertion and deletion operation in any position. Arrays require O(n)O(n) time to do the same thing, because you'd have to "shift" all the subsequent items over 1 index.
- Linked lists can continue to expand as long as there is space on the machine. Arrays (in low-level languages) must have their size specified ahead of time. Even if array is dynamic like array list that automatically resize themselves when they run out of space. the operation to resize a dynamic array has a large cost which can make a single insertion unexpectedly expensive.

Disadvantages of linked lists in Java:

- To access or edit an item in a linked list, you have to take O(i)O(i) time to walk from the head of the list to the iith item (unless of course you already have a pointer directly to that item). Arrays have constant-time lookups and edits to the iith item.

Q. How many types of Linked lists exists?

Java Linked lists can be categorized into three types:

Single linked: lists have a single pointer pointing to the next element in the list. The last pointer is empty or points to null, meaning the end of the list.

Doubly linked: lists have two pointers, one pointing to the next element and one pointing to the previous element.

The head node's previous pointer points to null and the tail node's next pointer points to null to signal the end of the list.

Circular linked: lists usually represent buffers. They have no head or tail, and the primary issue is avoiding infinite traversals because of the cycle.

Arrays are oftentimes a better substitute for a circular linked list, using the modulus operator to wrap around.

Q. Which interfaces are implemented by Linked List in Java?

Following interfaces are implemented by Java Linked lists

Serializable, Queue, List, Cloneable, Collection, Deque, Iterable

Q. What is the package name for Linked List class in Java?

`java.util`

Q. Java Linked List Vs Array?

Array	LinkedList
Performance slow	Performance Fast
Can Not Use Iterator	We Can Use Iterator
Array is non Synchronize	LinkedList is also Non Synchronized
Array Assumes every element is exactly the same size	Easier to store data of different sizes in linkedlist

Q. what is the difference between LinkedList And ArrayList ?

ArrayList	LinkedList
Time Complexity of ArrayList Get or retrieve data : $O(1)$ Add New Element : $O(n)$ Remove Element : $O(n)$	Time Complexity of LinkedList Get or retrieve data : $O(n)$ Add Element : $O(1)$ Remove Element : $O(1)$
ArrayList is the resizable array implementation of list interface	While LinkedList is the Doubly LinkedList implementation of the list interface
In ArrayList each index only hold the actual object	Memory Overhead in LinkedList is more as node in LinkedList needs to maintain the address of next and previous node

Q. what is the Set and Why Use Set Collection in java?

Basically, **Set** is a type of collection that does not allow duplicate elements. That means an element can only exist once in a Set. It models the set abstraction in mathematics.

Characteristics of a Set collection:

The following characteristics differentiate a Set collection from others in the Java Collections framework:

- Duplicate elements are not allowed.
- Elements are not stored in order. That means you cannot expect elements sorted in any order when iterating over elements of a Set.

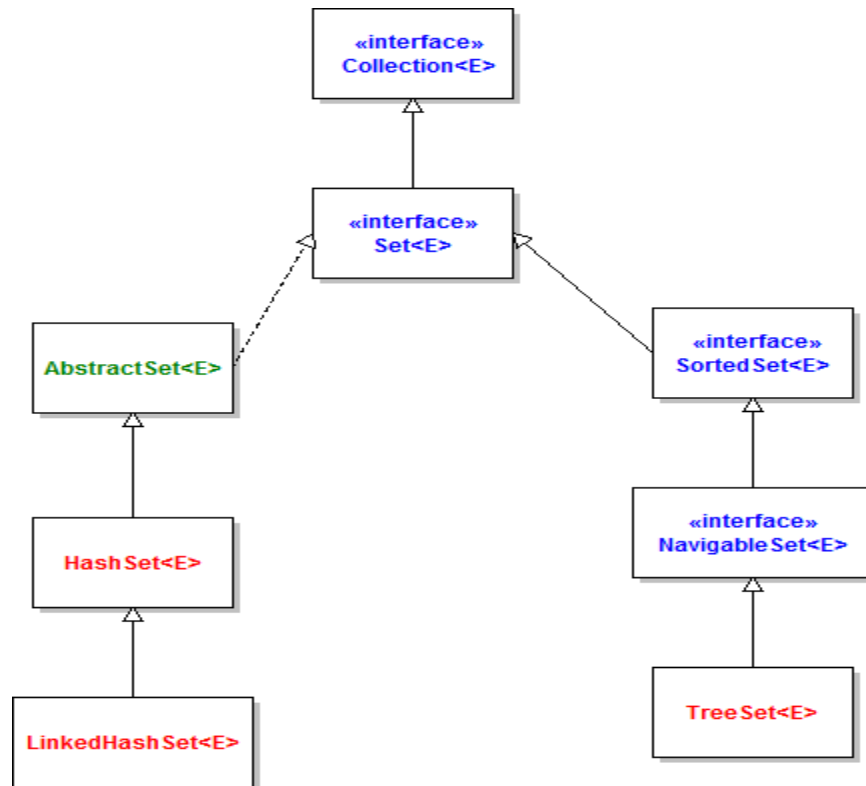
Why and When Use Sets?

Based on the characteristics, consider using a Set collection when:

- You want to store elements distinctly without duplication, or unique elements.
- You don't care about the order of elements.

For example, you can use a Set to store unique integer numbers; you can use a Set to store cards randomly in a card game; you can use a Set to store numbers in random order, etc.

The Java Collections Framework provides three major implementations of the Set interface: HashSet, LinkedHashSet and TreeSet. The Set API is described in the following diagram:



Let's look at the characteristics of each implementation in details:

- **HashSet:** is the best-performing implementation and is a widely-used Set implementation. It represents the core characteristics of sets: no duplication and unordered.
- **LinkedHashSet:** This implementation orders its elements based on insertion order. So consider using a LinkedHashSet when you want to store unique elements in order.
- **TreeSet:** This implementation orders its elements based on their values, either by their natural ordering, or by a Comparator provided at creation time.

Therefore, besides the uniqueness of elements that a Set guarantees, consider using HashSet when ordering does not matter; using LinkedHashSet when you want to order elements by their insertion order; using TreeSet when you want to order elements by their values.

Q. What is HashSet? What are the properties of HashSet object?

1. HashSet ensures uniqueness, In other words , every object in the HashSet presents only once
2. Unlike LinkedList, HashSet does not store the objects in orderly manner. It is possible that the object which we added first in the HashSet , may appear last in the output .
3. It offers constant time performance for basic operations like add, remove , contains and size assuming the hash function disperses the element properly among the buckets.
4. HashSet implementation is not synchronized.
 - * If multiple threads access a hash set concurrently, and at least one of the threads modifies the set, it must be synchronized externally.
 - *This is typically accomplished by synchronizing on some object that naturally encapsulates the set.

Q. What is the default initial capacity and initial load factor of HashSet object?

As we already discussed that HashSet internally uses HashMap. So the default initial capacity and initial load factor of HashSet is same as that of HashMap, that is

Default Initial Capacity of HashSet Object: 16

Initial Load Factor of HashSet Object: 0.75

Iteration performance of the HashSet object depends on the above two factors that is initial capacity and load factor:

- a. It is very important not to set the initial capacity too high or the load factor too low if iteration performance is important.

Q. What is the difference between HashSet and TreeSet?

Ordering: HashSet stores the object in random order . There is no guarantee that the element we inserted first in the HashSet will be printed first in the output .

For example

```
import java.util.HashSet;

public class HashSetExample {

    public static void main(String[] args) {

        HashSet<String> obj1= new HashSet<String>();

        obj1.add("Alive");

        obj1.add("is");

        obj1.add("Awesome");
```

```
        System.out.println(obj1);  
    }  
}
```

Elements are sorted according to the natural ordering of its elements in TreeSet. If the objects can not be sorted in natural order than use compareTo() method to sort the elements of TreeSet object

```
import java.util.TreeSet;
```

```
public class TreeSetExample {
```

```
    public static void main(String[] args) {  
        TreeSet<String> obj1= new TreeSet<String>();  
        obj1.add("Alive");  
        obj1.add("is");  
        obj1.add("Awesome");  
        System.out.println(obj1);  
    }  
}
```

2. Null value: HashSet can store null object while TreeSet does not allow null object. If one try to store null object in TreeSet object , it will throw Null Pointer Exception.

3. Performance : HashSet take constant time performance for the basic operations like add, remove contains and size. While TreeSet guarantees log(n) time cost for the basic operations (add,remove,contains).

4. Speed: HashSet is much faster than TreeSet,as performance time of HashSet is constant against the log time of TreeSet for most operations (add,remove ,contains and size) . Iteration performance of HashSet mainly depends on the load factor and initial capacity parameters.

5. Internal implementation : As we have already discussed How hashset internally works in java thus, in one line HashSet are internally backed by hashmap. While TreeSet is backed by a Navigable TreeMap.

6. Functionality : TreeSet is rich in functionality as compare to HashSet. Functions like pollFirst(),pollLast(),first(),last(),ceiling(),lower() etc. makes TreeSet easier to use than HashSet.

7. Comparision : HashSet uses equals() method for comparison in java while TreeSet uses compareTo() method for maintaining ordering .

Q. What Are The Similarities Between HashSet and TreeSet

1. Unique Elements : Since HashSet and TreeSet both implements Set interface . Both are allowed to store only unique elements in their objects. Thus there can never be any duplicate elements inside the HashSet and TreeSet objects.

2. Not Thread Safe : HashSet and TreeSet both are not synchronized or not thread safe. HashSet and TreeSet, both implementations are not synchronized. If multiple threads access a hash set/ tree set concurrently, and at least one of the threads modifies the set, it must be synchronized externally.

3. Clone() method copy technique: Both HashSet and TreeSet uses shallow copy technique to create a clone of their objects .

4. Fail-fast Iterators : The iterators returned by this class's method are fail-fast: if the set is modified at any time after the iterator is created, in any way except through the iterator's own remove method, the iterator will throw a ConcurrentModificationException. Thus, in the face of concurrent modification, the iterator fails quickly and cleanly, rather than risking arbitrary, non-deterministic behavior at an undetermined time in the future.

Q. When to prefer TreeSet over HashSet

1. Sorted unique elements are required instead of unique elements. The sorted list given by TreeSet is always in ascending order.

2. TreeSet has greater locality than HashSet.

If two entries are nearby in the order , then TreeSet places them near each other in data structure and hence in memory, while HashSet spreads the entries all over memory regardless of the keys they are associated to.

As we know Data reads from the hard drive takes much more latency time than data read from the cache or memory. In case data needs to be read from hard drive than prefer TreeSet as it has greater locality than HashSet.

3. TreeSet uses Red- Black tree algorithm underneath to sort out the elements. When one need to perform read/write operations frequently , then TreeSet is a good choice.

What is the LinkedHashSet?

LinkedHashSet collection can store the unique data in collection and manage the data sequence as per the user sequence.

The LinkedHashSet class of the Java collections framework provides functionalities of both the hashtable and the linked list data structure.

However, linked hash sets maintain a doubly-linked list internally for all of its elements. The linked list defines the order in which elements are inserted in hash tables.

In order to create a linked hash set, we must import the java.util.LinkedHashSet package first.

Once we import the package, here is how we can create linked hash sets in Java.

```
// LinkedHashSet with 8 capacity and 0.75 load factor
```

```
LinkedHashSet numbers = new LinkedHashSet(8, 0.75);
```

Notice, the part new LinkedHashSet(8, 0.75). Here, the first parameter is **capacity** and the second parameter is **loadFactor**.

- **capacity** - The capacity of this hash set is 8. Meaning, it can store 8 elements.
- **loadFactor** - The load factor of this hash set is 0.6. This means, whenever our hash table is filled by 60%, the elements are moved to a new hash table of double the size of the original hash table.

Q. what is the default capacity and load factor of LinkedHashSet in java Collection?

It's possible to create a linked hash set without defining its capacity and load factor. For example,

```
// LinkedHashSet with default capacity and load factor
```

```
LinkedHashSet<Integer> numbers1 = new LinkedHashSet<>();
```

By default,

- the capacity of the linked hash set will be 16
- the load factor will be 0.75

How to Create the LinkedHashSet by using other collection

If we want to create the LinkedHashSet by using other collection framework we have to create the object of collection class and store the element in it and we have to create the object of LinkedHashSet and pass the collection reference in its constructor.

```
import java.util.LinkedHashSet;
```

```
import java.util.ArrayList;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        // Creating an arrayList of even numbers
```

```
        ArrayList<Integer> evenNumbers = new ArrayList<>();
```

```
        evenNumbers.add(2);
```

```
        evenNumbers.add(4);
```

```
        System.out.println("ArrayList: " + evenNumbers);
```

```
        // Creating a LinkedHashSet from an ArrayList
```

```
        LinkedHashSet<Integer> numbers = new LinkedHashSet<>(evenNumbers);
```

```
        System.out.println("LinkedHashSet: " + numbers);
```

```
    }
```

```
}
```

Output:

ArrayList: [2, 4]

LinkedHashSet: [2, 4]

Q. what is the diff between HashSet and LinkedHashSet in Java?

Internal Working	HashSet internally uses HashMap for storing objects	LinkedHashSet uses LinkedHashMap internally to store objects
When To Use	If you don't want to maintain insertion order but want to store unique objects	If you want to maintain the insertion order of elements then you can use LinkedHashSet
Order	HashSet does not maintain insertion order	LinkedHashSet maintains the insertion order of objects
Complexity of Operations	HashSet gives O(1) complexity for insertion, removing, and retrieving objects	LinkedHashSet gives insertion, removing, and retrieving operations performance in order O(1).
Performance	The performance of HashSet is better when compared to LinkedHashSet and TreeSet.	The performance of LinkedHashSet is slower than TreeSet. It is almost similar to HashSet but slower because LinkedHashSet internally maintains LinkedList to maintain the insertion order of elements
Compare	HashSet uses equals() and hashCode() methods to compare the objects	LinkedHashSet uses equals() and hashCode() methods to compare it's objects
Null Elements	HashSet allows only one null value.	LinkedHashSet allows only one null value.

Q. what are the similarities between HashSet and LinkedHashSet?

- Duplicates: HashSet, LinkedHashSet and TreeSet are implements Set interface, so they are not allowed to store duplicates objects.
- Thread-safe: If we want to use HashSet, LinkedHashSet, and TreeSet in a multi-threading environment then first we make it externally synchronized because both LinkedHashSet and TreeSet are not thread-safe.
- All three are Cloneable and Serializable

Q. When to use HashSet, TreeSet, and LinkedHashSet in Java:

1. **HashSet:** If you don't want to maintain insertion order but wants to store unique objects.
2. **LinkedHashSet:** If you want to maintain the insertion order of elements then you can use LinkedHashSet.
3. **TreeSet:** If you want to sort the elements according to some Comparator then use TreeSet.

Q. Explain the All Set Operations With Examples?

Union of Sets: This operation adds all the elements in one set with the other.

To perform the union between two sets, we can use the `addAll()` method.

Now consider we have the two sets and we want to perform the union operation on it.

Let set1 = [1, 3, 2, 4, 8, 9, 0] and set2 = [1, 3, 7, 5, 4, 0, 7, 5]

After union we have the following result

Union = [0, 1, 2, 3, 4, 5, 7, 8, 9]

Example

```
import java.util.*;

public class SetExample {

    public static void main(String args[])
```

```
{  
    Set<Integer> a = new HashSet<Integer>();  
    a.addAll(Arrays.asList( new Integer[] { 1, 3, 2, 4, 8, 9, 0 }));  
  
    // Again declaring object of Set class  
    // with reference to HashSet  
    Set<Integer> b = new HashSet<Integer>();  
    b.addAll (Arrays.asList(new Integer[] { 1, 3, 7, 5, 4, 0, 7, 5 }));  
  
    // To find union  
    Set<Integer> union = new HashSet<Integer>(a);  
    union.addAll(b);  
    System.out.print("Union of the two Set");  
    System.out.println(union);  
}  
}
```

Intersection of Sets

This operation returns all the common elements from the given two sets. For the above two sets, the intersection would be

Let set1 = [1, 3, 2, 4, 8, 9, 0] and set2 = [1, 3, 7, 5, 4, 0, 7, 5]

After Intersection: [0, 1, 3, 4]

Example

```
import java.util.*;  
  
public class SetExample {  
    public static void main(String args[])  
    { Set<Integer> a = new HashSet<Integer>();
```

```
a.addAll(Arrays.asList( new Integer[] { 1, 3, 2, 4, 8, 9, 0 }));  
  
// Again declaring object of Set class  
  
// with reference to HashSet  
  
Set<Integer> b = new HashSet<Integer>();  
  
b.addAll(Arrays.asList(new Integer[] { 1, 3, 7, 5, 4, 0, 7, 5 }));  
  
// To find intersection  
  
Set<Integer> intersection = new HashSet<Integer>(a);  
  
intersection.retainAll(b);  
  
System.out.print("Intersection of the two Set");  
  
System.out.println(intersection);  
  
}  
}
```

Difference: This operation removes all the values present in one set from the other set.

To calculate the difference between the two sets, we can use the `removeAll()` method.

Let set1 = [1, 3, 2, 4, 8, 9, 0] and set2 = [1, 3, 7, 5, 4, 0, 7, 5]

After Difference: [2, 8, 9]

Example

```
import java.util.*;  
  
public class SetExample {  
  
    public static void main(String args[])  
  
    { Set<Integer> a = new HashSet<Integer>();  
  
        a.addAll(Arrays.asList( new Integer[] { 1, 3, 2, 4, 8, 9, 0 }));  
  
        // Again declaring object of Set class  
  
        // with reference to HashSet  
  
        Set<Integer> b = new HashSet<Integer>();
```

```
b.addAll(Arrays.asList(new Integer[] { 1, 3, 7, 5, 4, 0, 7, 5 }));  
  
// To find the symmetric difference  
  
Set<Integer> difference = new HashSet<Integer>(a);  
  
difference.removeAll(b);  
  
System.out.print("Difference of the two Set");  
  
System.out.println(difference);  
  
}  
}
```

Q. what is the difference between LinkedHashSet VS HashSet?

LinkedHashSet Vs. TreeSet

Here are the major differences between LinkedHashSet and TreeSet:

- The TreeSet class implements the SortedSet interface. That's why elements in a tree set are sorted. However, the LinkedHashSet class only maintains the insertion order of its elements.
- A TreeSet is usually slower than a LinkedHashSet. It is because whenever an element is added to a TreeSet, it has to perform the sorting operation.
- LinkedHashSet allows the insertion of null values. However, we cannot insert a null value to TreeSet

Q. What is TreeSet?

TreeSet is like HashSet which contains the unique elements only but in a sorted manner. The major difference is that TreeSet provides a total ordering of the elements. The elements are ordered using their natural ordering, or by a Comparator typically provided at sorted set creation time. The set's iterator will traverse the set in ascending element order.

Q. Why and when we use TreeSet?

We prefer TreeSet in order to maintain the unique elements in the sorted order .

Q. What is natural ordering in TreeSet?

"Natural" ordering is the ordering implied by the implementation of Comparable interface by the objects in the TreeSet. Essentially RBTree must be able to tell which object is smaller than other object, and there are two ways to supply that logic to the RB Tree implementation :

1. We need to implement the Comparable interface in the class(es) used as objects in TreeSet.
2. Supply an implementation of the Comparator would do comparing outside the class itself.

Q. Why do we need TreeSet when we already had Sorted Set?

SortedSet is an interface while TreeSet is the class implementing it. As we know, in java, we can not create the objects of the interface. The class implementing the interface must fulfill the contract of interface, i.e, concrete class must implement all the methods present in the interface. TreeSet is such an implementation.

Q. What happens if the TreeSet is concurrently modified while iterating the elements?

The iterator's returned by the TreeSet class iterator method are fail-fast. fail-fast means if the set is modified at any time after the iterator is created, in any way except the iterator's own remove method, the iterator will throw a ConcurrentModificationException. Thus, in the face of concurrent modification, the iterator fails quickly and cleanly

Q. Which copy technique (deep or shallow) is used by TreeSet clone() method?

According to Oracle docs, clone() method returns the shallow copy of the TreeSet instance. In shallow copy, Both objects A and B shared the same memory location.

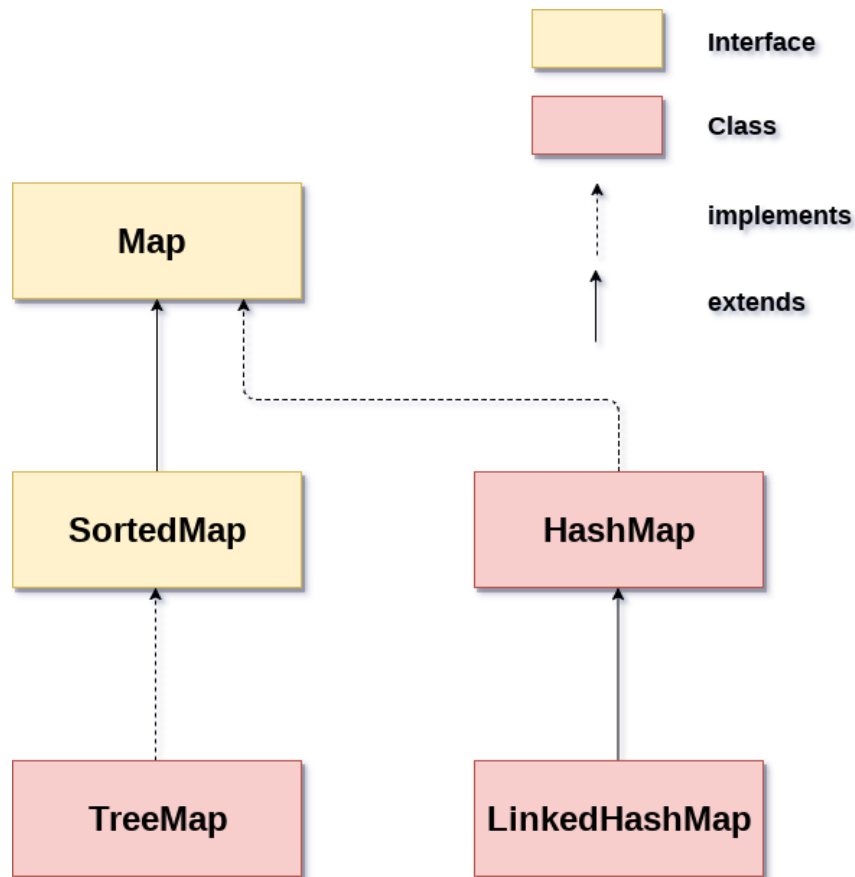
Q. How to convert HashSet to TreeSet object?

```
Set treeObject = new TreeSet (hashSetObject);
```

Q. What Is the Map interface in java?

Map interface is used for to store the data in the form of key and value pair. Key is unique identity of data and value is actual data present in map means we cannot add the duplicate key in map but we can add the duplicate value in map interface. In short we can say Map is combination of Set and List Collection Set represent key and List represent value.

Q. Explain the Map Hierarchy?



In the above Hierarchy we have the Map and SortedMap are the interfaces and HashMap and TreeMap are the implementer classes of above interfaces.

Q. Explain the Methods of Map interface

Map interface contain the some methods those help us to store and retrieve, delete, update etc operation on map interface

Some of the method given below with its example

void put(Object key, Object value): this method is used for to store data in map in the form of key and value pair.

Example

```
HashMap h = new HashMap();  
h.put(1, "Ram");  
h.put(2, "Shyam");  
h.put(3, "Dinesh");
```

key	Value
1	Ram
2	Shyam
3	Dinesh

This method is used for store the data in map using key and value pair shown in diagram

Object get (Object key): this method is used for retrieve data from map using its key if key not found return null

Note: Normally we use this method for retrieve data using its key as well as this method help us to find or search data from map using its key shown in following example.

Example of Fetch Data from Map using its key

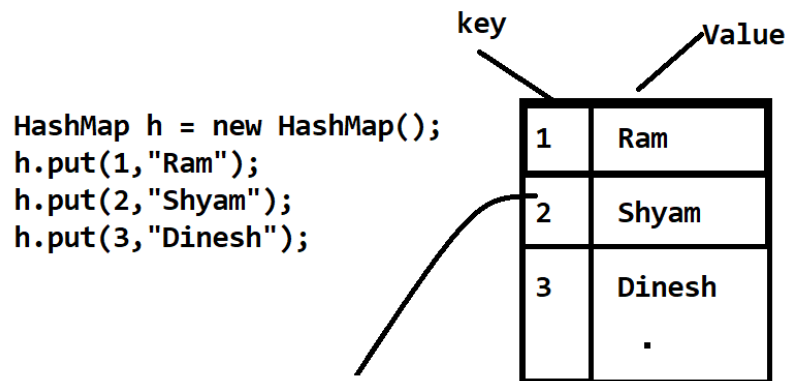
```
HashMap h = new HashMap();  
h.put(1, "Ram");  
h.put(2, "Shyam");  
h.put(3, "Dinesh");
```

key	Value
1	Ram
2	Shyam
3	Dinesh

Shyam
Object obj = h.get(2);
System.out.println(obj); //Shyam

Note: in this example we pass the key get() method i.e 2 so we get its value Shyam

Example of Search Element from Map using get() method



```
HashMap h = new HashMap();  
h.put(1, "Ram");  
h.put(2, "Shyam");  
h.put(3, "Dinesh");
```

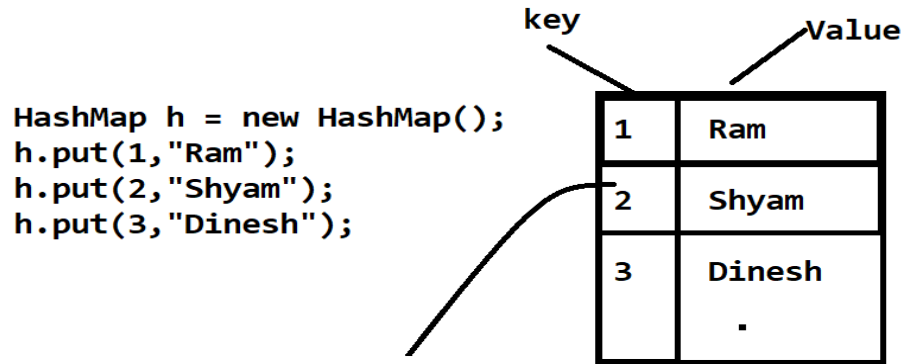
```
Object obj = h.get(2);
```

```
if(obj!=null)  
{ System.out.println("Element found");  
}  
else{  
    System.out.println("Element not found");  
}
```

Note: get() method return null value if key not present in map
and we check the condition obj!=null means if data found

boolean containsKey(Object key): this method check key present in map or not if present return true otherwise return false.

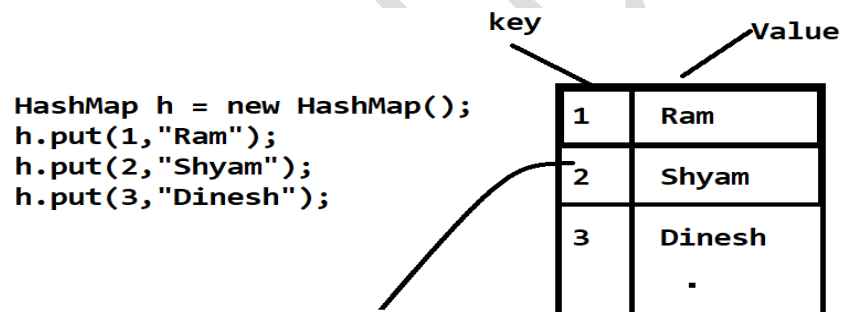
Example



```
HashMap h = new HashMap();
h.put(1, "Ram");
h.put(2, "Shyam");
h.put(3, "Dinesh");
```

```
boolean b= h.containsKey(2);
if(b)
{ System.out.println("Key present in map");
}
else{
    System.out.println("key not present in map");
}
```

Object remove (Object key): this method can delete the data from Map using its key if key found the delete the element and return deleted value if key not found then return null value.

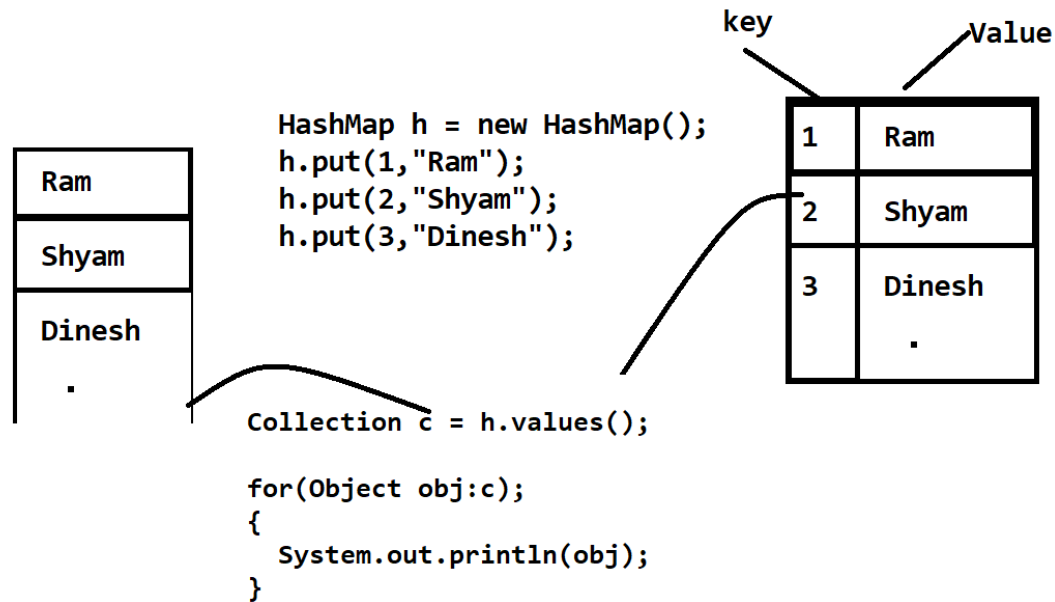


```
HashMap h = new HashMap();
h.put(1, "Ram");
h.put(2, "Shyam");
h.put(3, "Dinesh");
```

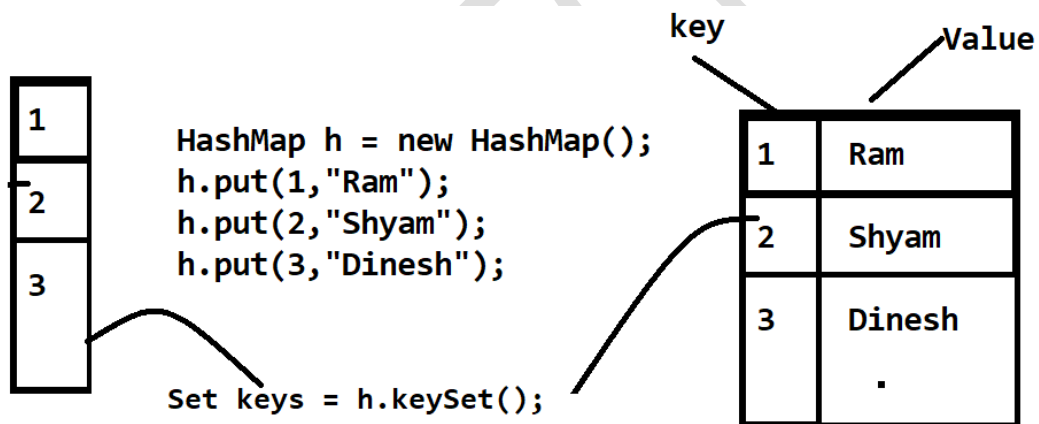
```
Object value= h.remove(2);

if(value!=null)
{
    System.out.println("Value deleted success");
}
else{
    System.out.println("Value not deleted");
}
```

Collection values(): this method can return only values from Map interface



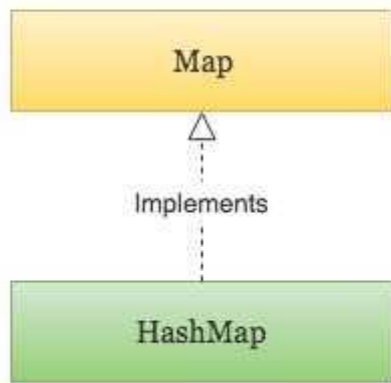
Set keyset(): this method return the all keys from Map interface and store in Set Collection.



Q. Explain the HashMap?

Java HashMap is a hash table based implementation of Java's Map interface. A Map, as you might know, is a collection of key-value pairs. It maps keys to values.

Java HashMap Class Hierarchy



Following are few key points to note about HashMap in Java -

- A HashMap cannot contain duplicate keys.
- Java HashMap allows null values and the null key.
- HashMap is an unordered collection. It does not guarantee any specific order of the elements.
- Java HashMap is not thread-safe. You must explicitly synchronize concurrent modifications to the HashMap.

How to create the HashMap

```
import java.util.HashMap;
import java.util.Map;

public class CreateHashMapExample {

    public static void main(String[] args) {

        // Creating a HashMap

        Map numberMapping = new HashMap();

        // Adding key-value pairs to a HashMap

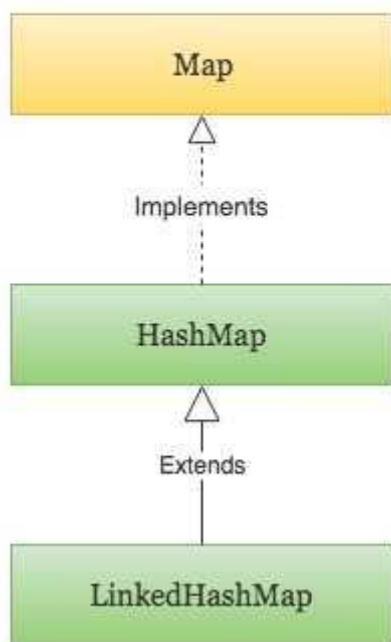
        numberMapping.put("One", 1);
```

```
numberMapping.put("Two", 2);  
  
numberMapping.put("Three", 3);  
  
numberMapping.putIfAbsent("Four", 4);  
  
System.out.println(numberMapping);  
  
}  
  
}
```

Q. Explain the LinkedHashMap In Java

Java LinkedHashMap is a hash table and doubly linked List based implementation of Java's Map interface. It extends the HashMap class which is another very commonly used implementation of the Map interface.

Java LinkedHashMap Class Hierarchy



The iteration order in a LinkedHashMap is normally the order in which the elements are inserted. However, it also provides a special constructor using which you can change the iteration order from the least-recently accessed element to the most-recently accessed element and vice versa. This kind of iteration order can be useful in building LRU caches. In any case, the iteration order is predictable.

Following are some important points to note about LinkedHashMap in Java -

- A LinkedHashMap cannot contain duplicate keys.
- LinkedHashMap can have null values and the null key.
- Unlike HashMap, the iteration order of the elements in a LinkedHashMap is predictable.
- Just like HashMap, LinkedHashMap is not thread-safe. You must explicitly synchronize concurrent access to a LinkedHashMap in a multi-threaded environment.

Creating and Initializing a LinkedHashMap

```
import java.util.LinkedHashMap;

public class CreateLinkedHashMapExample {

    public static void main(String[] args) {

        // Creating a LinkedHashMap

        LinkedHashMap wordNumberMapping = new LinkedHashMap();

        // Adding new key-value pairs to the LinkedHashMap

        wordNumberMapping.put("one", 1);

        wordNumberMapping.put("two", 2);

        wordNumberMapping.put("three", 3);

        wordNumberMapping.put("four", 4);
```

```
        wordNumberMapping.putIfAbsent("five", 5);

        System.out.println(wordNumberMapping);
    }
}
```

Q. Perform Following Operation on LinkedHashMap

- Check if a key exists in a LinkedHashMap.
- Check if a value exists in the LinkedHashMap.
- Modify the value associated with a given key in the LinkedHashMap.

Example

```
import java.util.LinkedHashMap;

public class AccessEntriesFromLinkedHashMapExample {

    public static void main(String[] args) {

        LinkedHashMap<Integer, String> customerIdNameMapping = new LinkedHashMap<>();

        customerIdNameMapping.put(1001, "Jack");
        customerIdNameMapping.put(1002, "David");
        customerIdNameMapping.put(1003, "Steve");
        customerIdNameMapping.put(1004, "Alice");
        customerIdNameMapping.put(1005, "Marie");

        System.out.println("customerIdNameMapping : " + customerIdNameMapping);

        Integer id = 1005;

        if(customerIdNameMapping.containsKey(id)) {
```

```
        System.out.println("Found the customer with id " + id + " : " + customerIdNameMapping.get(id));
    } else {
        System.out.println("Customer with id " + id + " does not exist");
    }

    String name = "David";

    if(customerIdNameMapping.containsValue(name)) {
        System.out.println("A customer named " + name + " exist in the map");
    } else {
        System.out.println("No customer found with name " + name + " in the map");
    }

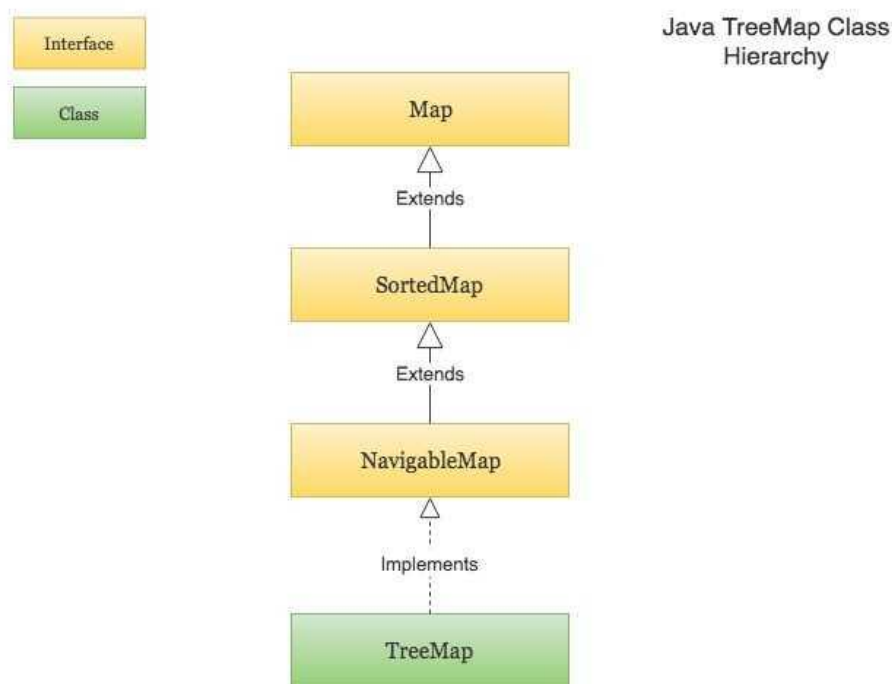
    customerIdNameMapping.put(id, "Bob");

    System.out.println("Changed the name of customer with id " + id + ", New mapping : " +
        customerIdNameMapping);
}
}
```

Q. Explain the TreeMap in Detail

Java TreeMap is a Red-Black tree based implementation of Java's Map interface.

The entries in a TreeMap are always sorted based on the natural ordering of the keys, or based on a custom Comparator that you can provide at the time of creation of the TreeMap. The TreeMap class is part of Java's collection framework. It implements the NavigableMap interface, which in turn extends the SortedMap interface. Following is the class hierarchy of TreeMap.



The SortedMap interface provides functionalities to maintain the ordering of keys. And the NavigableMap interface provides functionalities to navigate through the map. For example, finding the entry just greater than or just less than the given key, finding the first and last entry in the TreeMap etc.

Since a TreeMap implements NavigableMap interface, it has the functionalities of both the NavigableMap as well as the SortedMap.

Following are few key points to note about TreeMap in Java -

- A TreeMap is always sorted based on keys. The sorting order follows the natural ordering of keys. You may also provide a custom Comparator to the TreeMap at the time of creation to let it sort the keys using the supplied Comparator.
- A TreeMap cannot contain duplicate keys.
- TreeMap cannot contain the null key. However, It can have null values.
- TreeMap is not synchronized. Access to TreeMaps must be synchronized explicitly in a multi-threaded environment.

Creating a TreeMap

```
import java.util.SortedMap;

import java.util.TreeMap;

public class CreateTreeMapExample {

    public static void main(String[] args) {

        // Creating a TreeMap

        SortedMap fileExtensions = new TreeMap();

        // Adding new key-value pairs to a TreeMap

        fileExtensions.put("python", ".py");

        fileExtensions.put("c++", ".cpp");

        fileExtensions.put("kotlin", ".kt");

        fileExtensions.put("golang", ".go");

        fileExtensions.put("java", ".java");

        // Printing the TreeMap (Output will be sorted based on keys)

        System.out.println(fileExtensions);

    }

}
```